

Podcast Search with Retrieval-Augmented Generation

A semantic retrieval + LLM pipeline for long-form audio transcripts

Daniel Ruijs • Pavlos Spanoudakis • Bhaswat Raj • Parniansadat Razavipour



Problem and approach

The core challenges

1. Podcasts are long, unstructured, and often cover several topics in one episode.
2. Query wording may differ from transcript wording.
3. Users often want a specific relevant moment or answer, not an entire episode.

1

Fine-grained retrieval

Search inside episodes using segment-level retrieval rather than only metadata or full episodes.

2

Hybrid retrieval

Use embeddings so meaning matters, not just exact words. Combine lexical and semantic signals to find better matches.

3

Grounded generation

Use retrieved transcript chunks to summarize, highlight, and answer questions.

Choosing the embedding model

Three-stage filtering pipeline

● 1. Efficiency screen

Measured encoding latency on a laptop GPU to keep full-corpus indexing practical.

● 2. Semantic separation tests

Compared intra-group vs inter-group cosine similarity on topical and synonym datasets.

● 3. Small-index retrieval evaluation

Ran vector retrieval test on 3,000 chunks and judged top-10 results with an LLM.
LLM judge (3 runs) → averaged relevance scores

Selected model: sentence-transformers/all-mpnet-base-v2
best semantic separation and best retrieval quality, even if it is slightly slower than the smallest alternatives.

Table 1: Embedding model efficiency benchmark

Model	Params	Avg (ms)
cointegrated/rubert-tiny2	29M	5.3
sentence-transformers/all-MiniLM-L6-v2	23M	6.8
BAAI/bge-small-en-v1.5	33M	12.2
intfloat/multilingual-e5-small	118M	15.7
sentence-transformers/all-mpnet-base-v2	109M	16.7

Table 2: Semantic similarity evaluation on topical clustering dataset

Model	Intra ↑	Inter ↓	Sep ↑	Encode (s)
sentence-transformers/all-mpnet-base-v2	0.686	0.266	0.420	0.36
sentence-transformers/all-MiniLM-L6-v2	0.609	0.303	0.306	0.07
BAAI/bge-small-en-v1.5	0.833	0.660	0.174	0.37
intfloat/multilingual-e5-small	0.915	0.871	0.044	0.32
cointegrated/rubert-tiny2	0.958	0.922	0.036	0.07

Table 3: Semantic similarity evaluation on synonym generalisation dataset

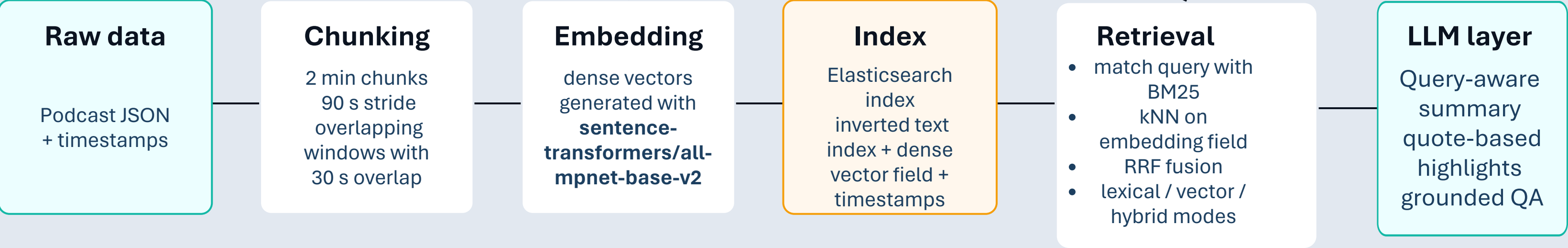
Model	Intra ↑	Inter ↓	Sep ↑	Encode (s)
sentence-transformers/all-mpnet-base-v2	0.537	0.265	0.272	0.041
sentence-transformers/all-MiniLM-L6-v2	0.445	0.259	0.186	0.023
BAAI/bge-small-en-v1.5	0.866	0.816	0.050	0.229
intfloat/multilingual-e5-small	0.854	0.836	0.018	0.029
cointegrated/rubert-tiny2	0.876	0.870	0.007	0.018

Table 4: Embedding model retrieval performance on the small index (3000 chunks, top-10 evaluation).

Model	P@10	MRR	nDCG@10	Time
all-mpnet-base-v2	0.646	0.888	0.849	9m 31.5s
bge-small-en-v1.5	0.549	0.796	0.766	3m 6.7s
all-MiniLM-L6-v2	0.541	0.779	0.764	42.65s

System architecture reused

The Spotify Podcasts Dataset



Reused components

Hugging Face model · Elasticsearch · Gemini SDK · Ollama
judge

keep retrieval grounded in the transcript, then let the LLM operate only on the retrieved evidence.

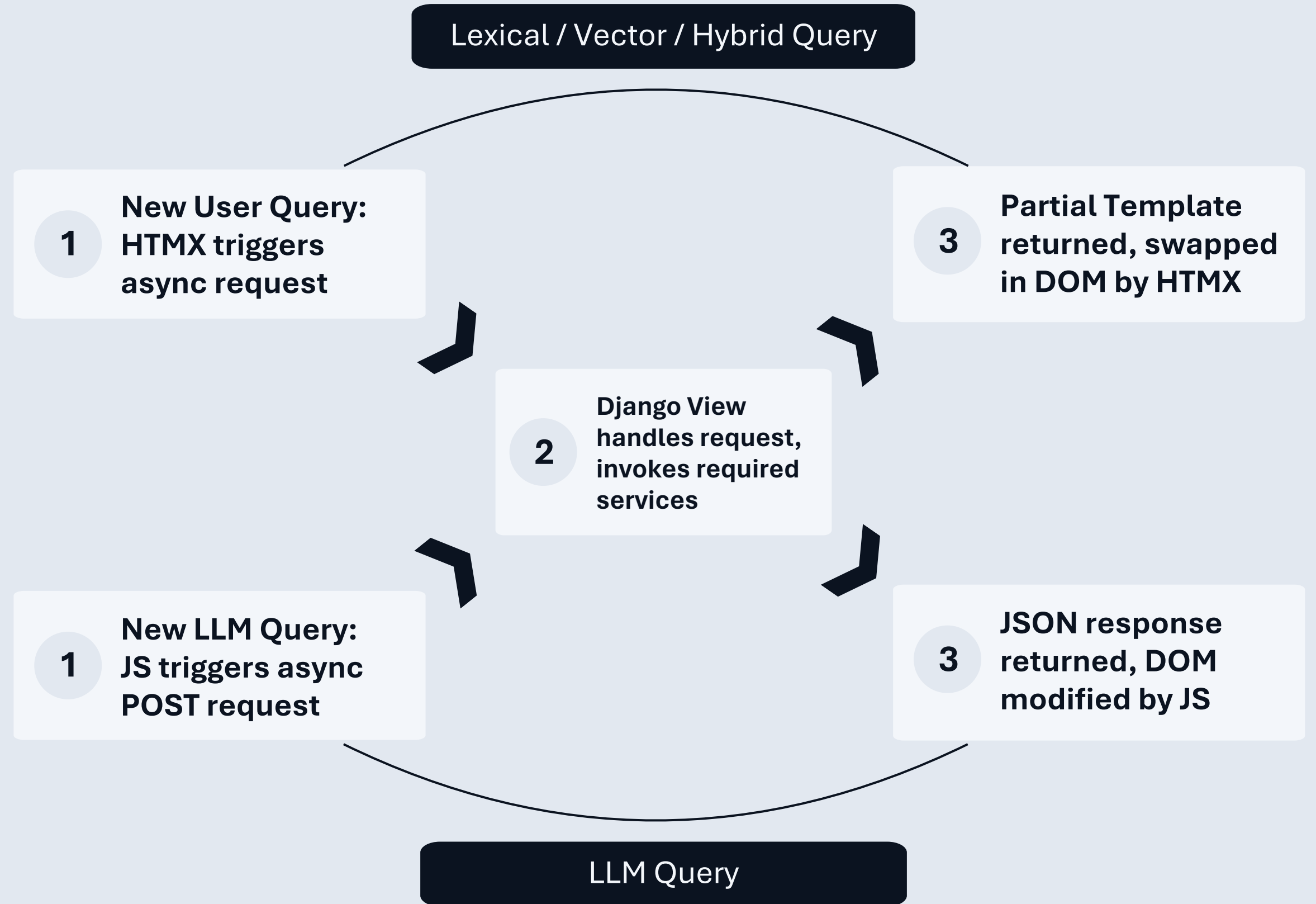
Engine architecture

Backend

- **Django** server: **views** process requests, **services** hold app logic
- Embedding model used via **sentence_transformers**
- Communication with Elasticsearch instance using **elasticsearch** lib
- Usage of LLM via **google.genai**

Frontend

- Frontend served by Django, no separate application needed
- Page rendered using Django **templates** along with **CSS + JS** assets
- **HTMX** handles partial UI updates after new query invocations
- JS used to invoke LLM services via **fetch()**, and update UI by DOM manipulation



Retrieval: vector search, BM25, and RRF fusion

The user query is processed in parallel by two retrievers.

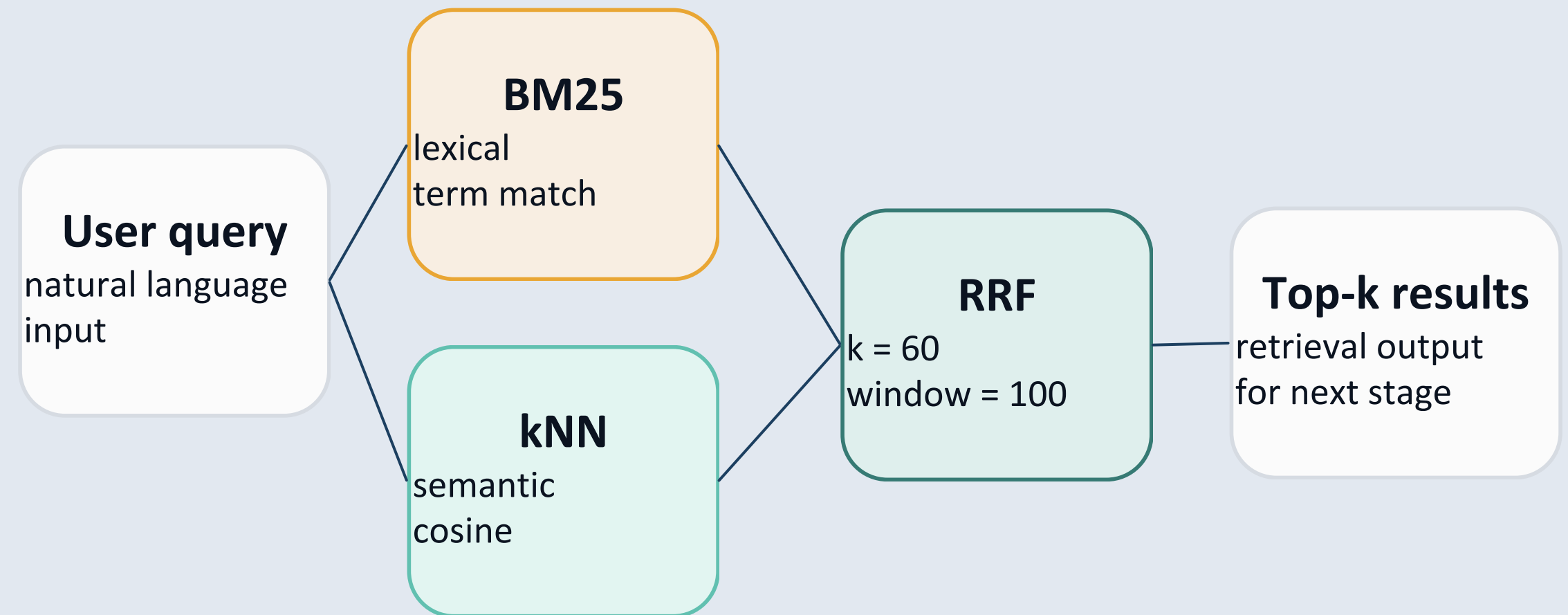
Vector branch: embed query with the same MPNet model and run kNN using cosine similarity.

Lexical branch: run a BM25 match query over transcript text.

Hybrid mode fuses both ranked lists with Reciprocal Rank Fusion (RRF).

$$\text{RRF}(d) = \sum_i \frac{1}{k + \text{rank}_i(d)}$$

Settings used in Elasticsearch: rank constant $k = 60$, rank window size = 100.



the system can run in lexical-only, vector-only, or hybrid mode. Hybrid is not a separate model; it is rank fusion over the two retrieval outputs.

LLM integration after retrieval

Model used in the system: Gemini 3.1 Flash Lite, with fallback to Gemini 2.5 Flash Lite. Structured outputs are enforced through the Google GenAI SDK + Pydantic / JSON Schema.

Query-aware summarization

Top-k chunks are concatenated into one context block; the prompt tells the model to summarize only what matters for the search query.

Highlight extraction

Instead of fragile character spans, the model returns short quoted substrings that can be mapped back to transcript text.

Retrieval-augmented QA

The user asks a question after retrieval, and the model answers only from the selected chunks.

Feedback annotation

The LLM assigns 0–3 relevance labels to retrieved results so metrics such as P@K, MRR, and nDCG can be computed. This also uses structured output

Highlight extraction design

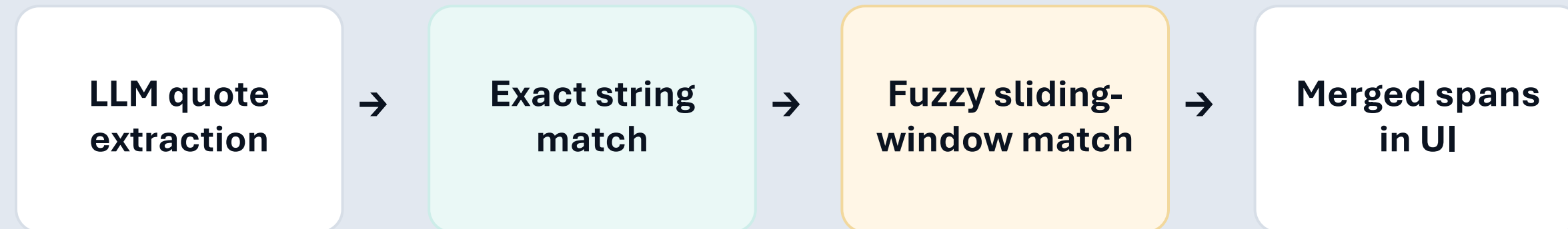
Integration via Google GenAI Python SDK with structured JSON outputs defined through schema/Pydantic.

Character offsets from the LLM were inconsistent, so the system switched to quote extraction instead of start/end span prediction. So the system switched to quote extraction instead of start/end span prediction.

Structured output

```
{"highlights": [  
  {"result_index": 0,  
   "quotes": ["phrase 1",  
             "phrase 2"]}  
]}
```

Batched in groups of 5 chunks to improve consistency and reduce prompt complexity.



Matching strategy

1. Try exact matching first.
2. If exact matching fails, compare the quote against sliding windows from the transcript.
3. Accept the best fuzzy match when similarity > 0.82 .
4. Merge overlapping spans before rendering highlights.

Evaluation protocol

Retrieval benchmark

- 20 synthetic queries spanning politics, science, business, health, culture, and sport
- Compare lexical, vector, and hybrid retrieval
- Evaluate at $K = 5, 10, 15, 20$
- Gemini 3.1 Flash Lite assigns 0–3 relevance labels
- Binary labels: 1 when relevance ≥ 2
0 when relevance < 2
- 0: not relevant, 1: briefly mentions the topic, 2: partially relevant with useful detail, 3: strongly relevant / exhaustive discussion

Fair nDCG comparator

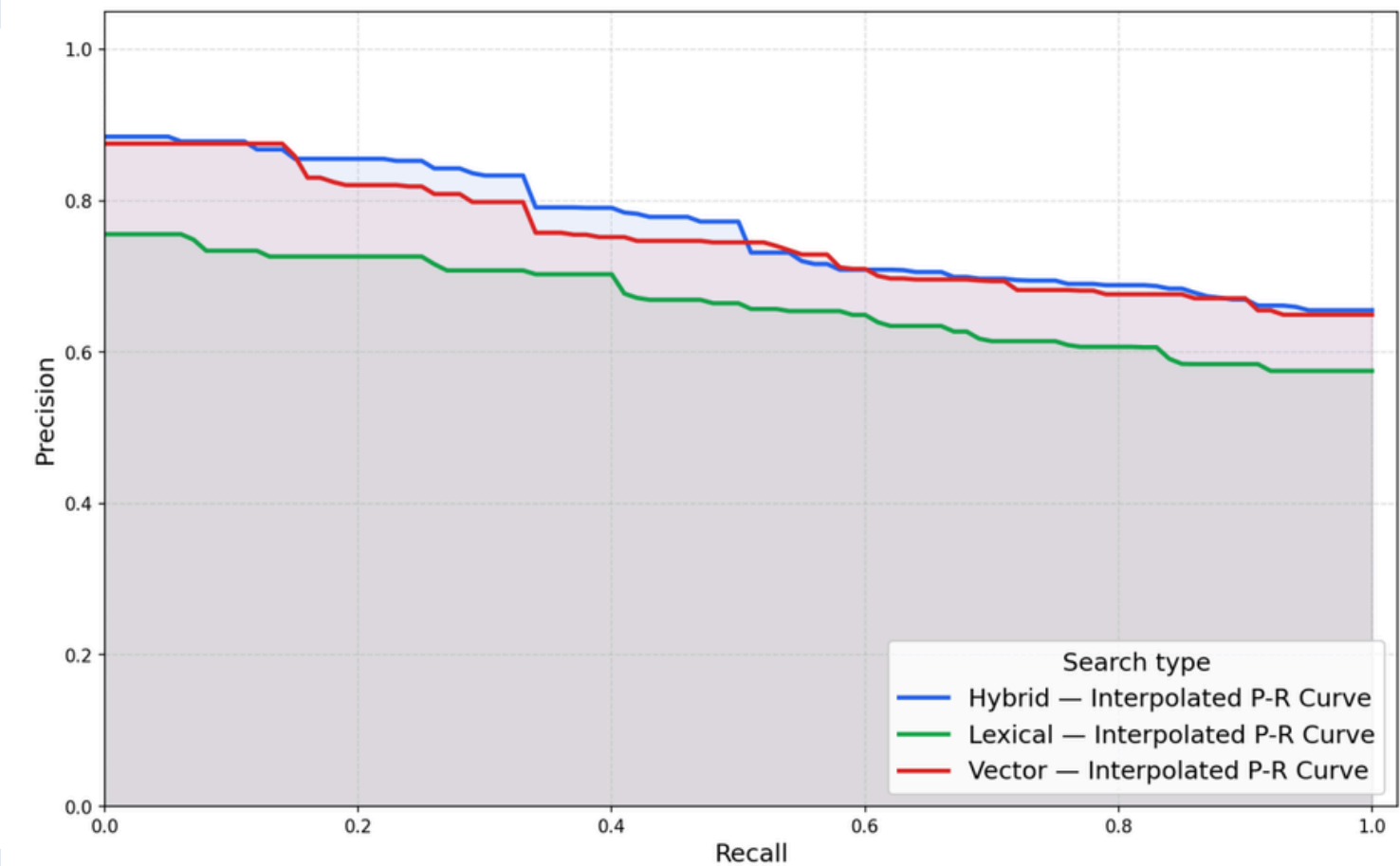
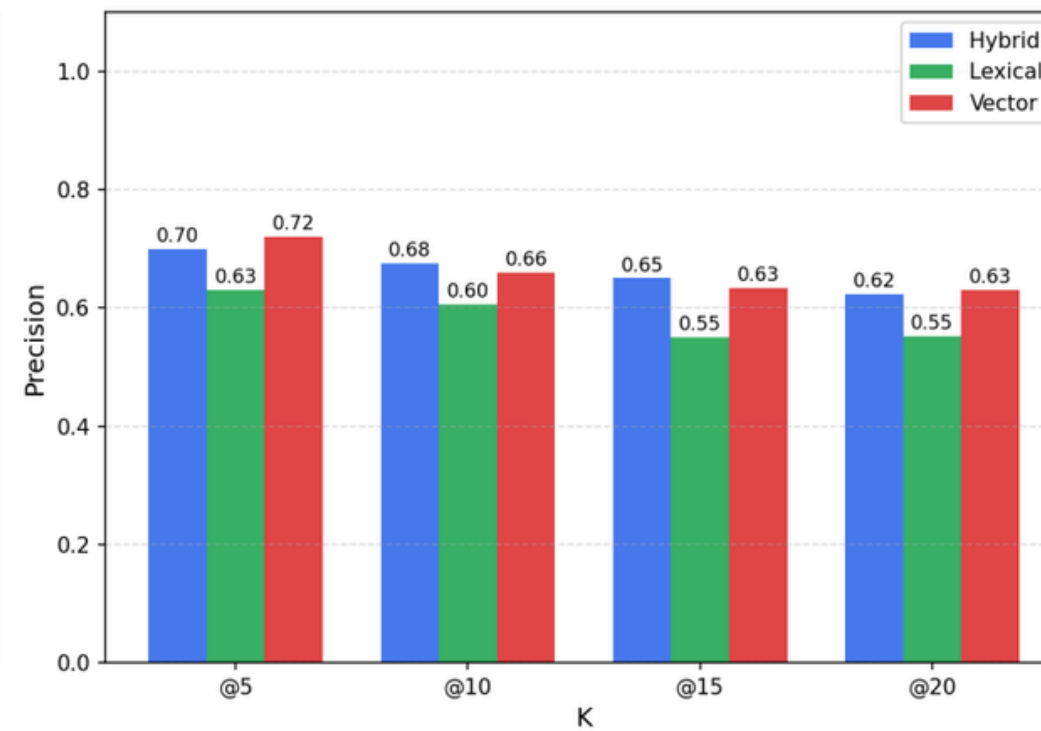
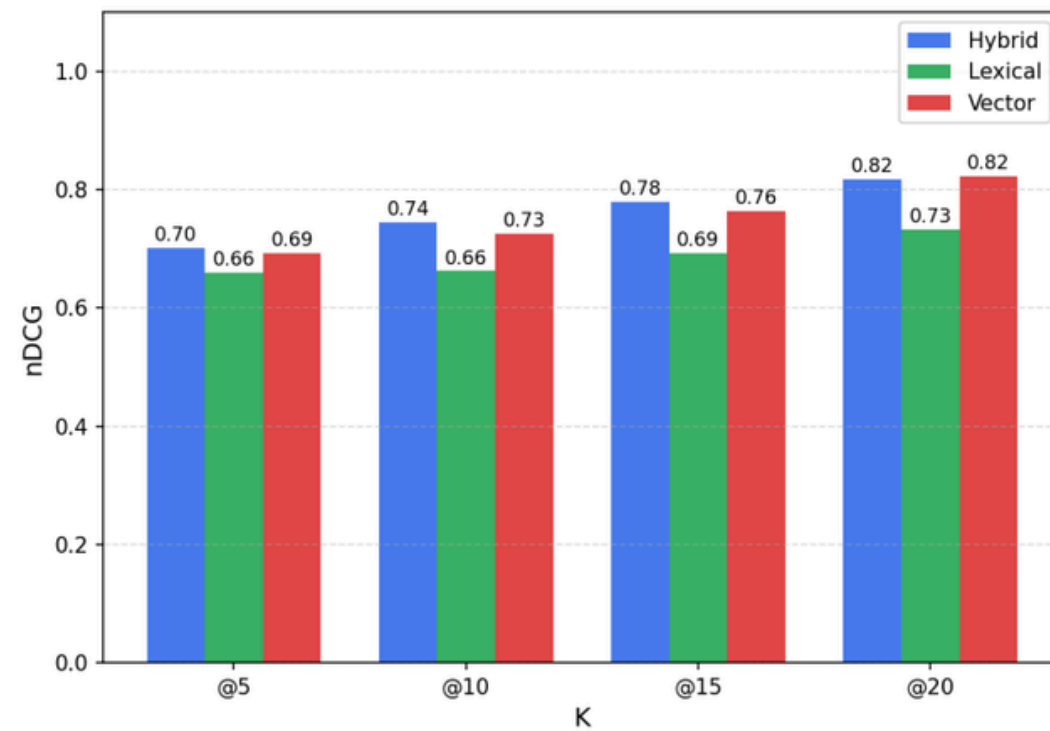
To enable a fair comparison, the nDCG ideal ranking is computed from the union of results returned by all three methods per query, deduplicated by episode and capped at 20.

Highlight benchmark

- Run the highlight model on the same retrieved chunks
- Claude Sonnet 4.6 rates quote quality on a 0–3 scale
- Positive highlight = quality ≥ 2
- Report Precision@K, Recall@K, F1@K, and AvgQuality@K

This setup is efficient for an end-to-end comparison, but it should be treated as indicative rather than definitive.

Main retrieval results



- Semantic retrieval clearly beats pure keyword search.
- Hybrid search is especially strong at shallow ranks (5/10/15) meaning it ranks more relevant documents higher.
- Vector search gives the best recall at K=20: 0.95.
- Vector search achieved the best overall nDCG@20: 0.822
- Hybrid search was almost tied at nDCG@20: 0.818
- Lexical search lagged on ranking quality: nDCG@20 = 0.733

Table 5: Retrieval comparison at $K = 20$ (20-query benchmark).

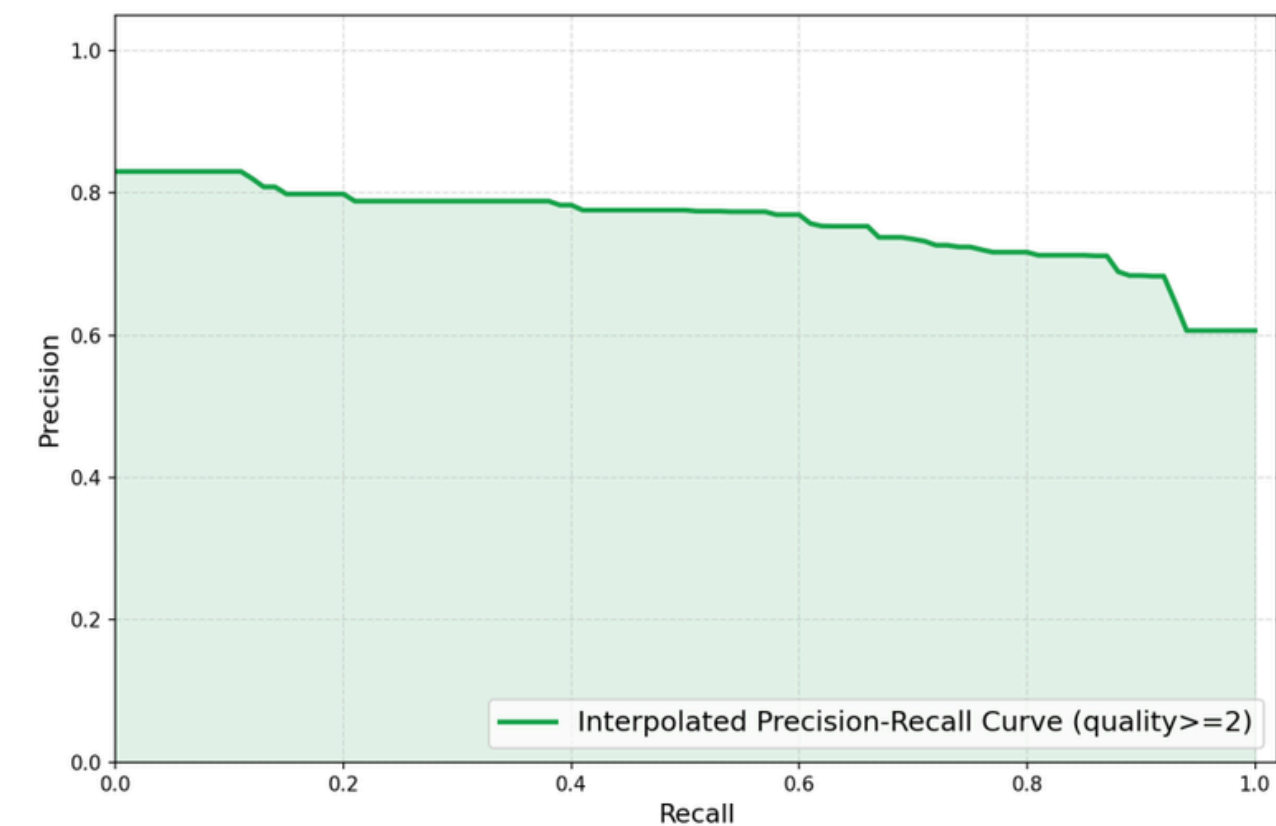
Search mode	Precision	Recall	DCG	nDCG
Hybrid	0.623	0.900	12.274	0.818
Lexical	0.553	0.900	11.205	0.733
Vector	0.630	0.950	12.191	0.822

LLM highlighting results

- Stable precision across K (0.67-0.68): Consistent highlight quality.
- Recall increases with K (0.47 $\rightarrow$ 0.94): More results capture more relevant content.
- Slight decrease in average highlight quality (1.61 $\rightarrow$ 1.54): lower-ranked results are less relevant $\rightarrow$ weaker context for generating high-quality highlights.
- Takeaway:
 - Small K for high-quality highlights
 - Large K for better coverage, but noisier highlights

Table 6: Averaged highlight-quality metrics (20-query benchmark).

K	Precision	Recall	F1	AvgQuality
5	0.678	0.466	0.501	1.610
10	0.684	0.695	0.665	1.596
15	0.675	0.855	0.739	1.568
20	0.669	0.937	0.768	1.542



Take-away message

- **1. RAG is a strong fit for podcast search**

Retrieval finds relevant transcript segments; generation makes them easier to use.

- **2. Semantic retrieval is the main win**

Semantic matching improves ranking quality, while hybrid is strong at shallow ranks.

- **3. LLM post-processing adds user value**

Summaries, highlights, and grounded Q&A make retrieved evidence more usable.

Future work: better segmentation, larger user-facing evaluation, and richer cross-episode reasoning.